Rapport TNL

Amaury JACOB-PIACENTINI Sharaine MALAVIJY

29 mars 2026

Table des matières

1 Introduction **2**

2 Analyse des données **2**
2.1 Classification de sentiments 2
2.1.1 Description du jeu de données 2
2.1.2 Analyse du jeu de données 2
2.2 Classification des discours de présidents 3
2.2.1 Description du jeu de données 3
2.2.2 Analyse du jeu de données 3
2.3 Prétraitement des données 4
2.3.1 Description 4
2.3.2 Objectifs du prétraitement 4
2.3.3 Nettoyage du texte 4

3 Classification **6**
3.1 Méthodologie 6
3.2 Modèles testés 6
3.2.1 Classification simple 6
3.2.2 Transformers 7
3.3 Résultats et analyse 7
3.3.1 Résultats sur les 3 classificateurs (SVM linéaire, MultinomialNB et LogisticRegression) 7
3.3.2 Transformer 8

4 Conclusion **8**

1 Introduction

Le but de ce projet est de classifier des textes afin de pouvoir nous former aux divers outils de traitement du langage naturel. Pour cela, nous avons deux jeux de données différents : un jeu de données contenant des critiques de films, et un autre contenant des discours de Jaques Chirac et de François Mitterrand.
Nous avons divisé notre travail en plusieurs parties. Dans un premier temps, nous avons fait de l'analyse des données et du pré traitement des données. Ensuite nous avons pu commencer le travail de classification. Nous avons exploré plusieurs techniques de classification vu lors des TME et du cours.

2 Analyse des données

2.1 Classification de sentiments

2.1.1 Description du jeu de données

Ce jeu de données contient des critiques de films. Il est équilibré, nous avons 2000 critiques de films au total dont 1000 critiques positives et 1000 critiques négatives. Chaque critique de film est constituée de plusieurs phrases. Nous avons également les labels associés à chaque critique de film : 1 pour les critiques positives et 0 pour les critiques négatives.

2.1.2 Analyse du jeu de données

Dans un premier temps, nous avons calculé la taille du vocabulaire du jeu de données. Il y a 39659 mots différents avant de faire un pré traitement des données.
Après avoir calculé la taille du vocabulaire, il a fallu regarder la fréquence des mots dans le dataset. Voici un exemple de la fréquence des mots dans le dataset :

Mot	Fréquence
the	76529
and	35576
of	34123
to	31937
is	25195

Table 1 – Mots les plus fréquents

Comme attendu, avant toute forme de pré traitement, les mots les plus fréquents sont les stops words. Ils n'apportent pas d'information discriminante plus tard dans la partie prétraitement, ils seront supprimés.
Une meilleure façon de visualiser la fréquence des mots est d'utiliser un word cloud. Ainsi que la distribution de Zipf.

and
of the
in the
in a
is a
but
film
the
to be
and the
on the
the movie
this film
with a
for the
to the
the most
as the
it is
that the
there i
the same
one of
that i
the first
to see
the character
from the
by the
this
a few
the world
play
movie
the movie
film i
he is
or
from
at the
of his
her
the best
which
their
to a
all the
that
the only
with his
has
some of
it was
work
this that
the two
well
would be
could have
be a
here
has been
over
one
that
out of
they are
who
else
there are
after
and his
have a
in his
them
then
into
however
not a
all
not
to get
if you
the way
like a
such a
being
another
by a
seems to
go
that
the
to
a great
a little
a film
a film
a film
a film
a film
work
to do
it sounds
it sounds
it sounds

Figure 3 – *
Word cloud des discours de présidents

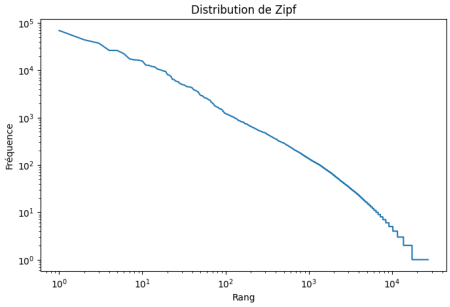

Figure 4 – *
Distribution de Zipf des discours de présidents

2.3 Prétraitement des données

2.3.1 Description

Le prétraitement des données est une des étapes les plus importantes dans ce projet. Nous avons plusieurs techniques différentes pour faire du prétraitement afin d'arriver à une pipeline de prétraitement qui nous semble la plus adaptée pour nos jeux de données.

2.3.2 Objectifs du prétraitement

Dans un premier temps il a fallu faire un préprétraitement des données. Nous avons d'abord converti tous les mots en minuscules afin de ne pas avoir de doublons dans notre vocabulaire. Ensuite, nous avons supprimé les caractères spéciaux et les chiffres car ils n'apportent pas d'informations pertinentes pour la classification.
Nous avons aussi essayé de garder uniquement les premières lignes de chaque critique de film pour voir si cela pouvait améliorer les performances de nos modèles. Cependant, cela n'a pas eu d'impact significatif sur les performances de nos modèles.

2.3.3 Nettoyage du texte

Pour le jeu de données des avis de films nous avons supprimé les mots les plus fréquents car ils sont très peu discriminants. Il a fallu aussi ajouter aux stop words une liste de mots plus rares mais non discriminants.

Les plus fréquents n'apportent pas d'informations discriminantes c'est à dire qu'ils sont présents dans les critiques de films positives et négatives.

Pour le jeu de données des discours de présidents nous avons supprimé les stop words francais, nous avons aussi retiré les mots "français", "pays" et "qu" car ils sont très peu discriminants. Ce data set est beaucoup plus complexe à traiter que le précédent. Comme il est déséquilibré les discoursd de Chirac sont sur représentées sur les graphes.

Il est intéressant de comparer les graphes de la répartition des mots positifs et négatifs avant et après le prétraitement des données.

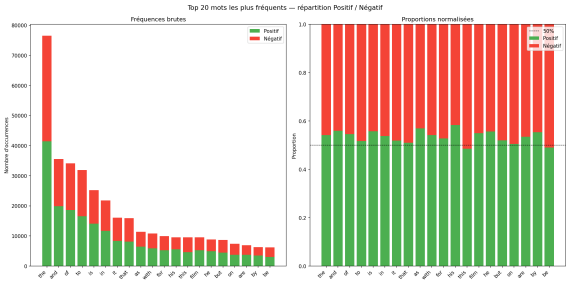

FIGURE 5 – *
fréquence des mots et caractère discriminant des mots dans les critiques de films avant prétraitement

FIGURE 6 – *
fréquence des mots et caractère discriminant des mots dans les critiques de films après prétraitement

Comme on peut les voir sur les graphes après le prétraitement des données, les mots les plus fréquents ont évidement changé mais surtout nous pouvons remarquer que les mots sont bien plus discriminants qu'avant le prétraitement des données. Nous avons beaucoup plus de mots qui sont présents dans les critiques de films positives que dans les critiques de films négatives et inversement.

Nous pouvons voir que la courbe de Zipf est beaucoup plus lisse après le prétraitement des données.

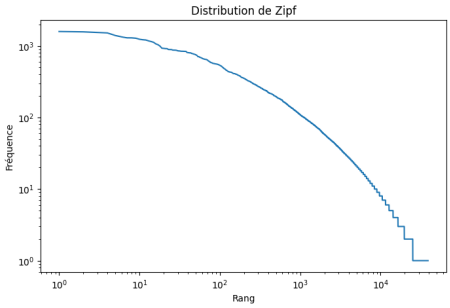

FIGURE 7 – *
Distribution de Zipf des critiques de films après prétraitement

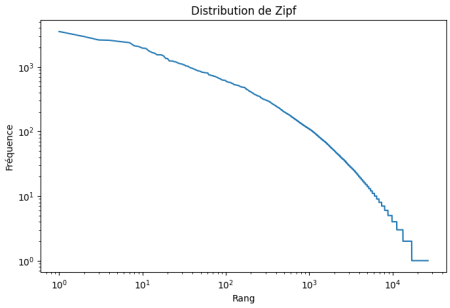

FIGURE 8 – *
Distribution de Zipf des discours de présidents après prétraitement

Sur les Figures 2 et 4 les courbes de zipf montrent que les corpus sont dominés par un petit nombre de mots très fréquents, tandis que la grande majorité des mots sont rares. Cependant, après le prétraitement des données, les courbes de Zipf deviennent moins linéaires, ce qui indique que les mots les plus fréquents ont été supprimés et que les mots restants sont plus équilibrés en termes de fréquence.

Ce prétraitement des données est le premier que nous avons implémenté. Il ne donnait pas de résultats satisfaisants donc nous nous sommes dirigés vers la TF-IDF pour faire du prétraitement des données.

La TF-IDF (Term Frequency – Inverse Document Frequency) consiste à attribuer un poids à chaque mot en fonction de son importance dans un document et dans l'ensemble du corpus. La composante TF (Term Frequency) mesure la fréquence d'un mot dans un document, tandis que la composante IDF (Inverse Document Frequency) pénalise les mots apparaissant fréquemment dans de nombreux documents. Ainsi, un mot fréquent dans un document mais rare dans le corpus aura un poids élevé, ce qui le rend plus discriminant pour la classification.

Nous avons utilisé la TF-IDF de la librairie Scikit-learn, beaucoup de configurations ont été

testées et nous nous sommes arrêtés sur celle qui donnait les meilleurs résultats.

Pour le jeu de données des films, nous avons utilisé les paramètres `max_df = 0.95`, `min_df = 0.005` et `sublinear_tf = True`. Le paramètre `min_df = 0.005` permet de supprimer les mots apparaissant uniquement dans 0.5% des documents, ce qui réduit le bruit lié aux mots très rares ou aux erreurs typographiques. Le paramètre `max_df = 0.95` signifie que nous filtrons explicitement les mots très fréquents.

L'option `sublinear_tf = True` applique une transformation logarithmique sur la fréquence des mots, donnée par $1 + \log(\text{tf})$, ce qui permet de limiter l'influence des mots très fréquents dans un document et d'obtenir une représentation plus équilibrée.

Enfin, nous avons choisi de désactiver le prétraitement manuel (mise en minuscules, suppression des accents, etc.), car l'implémentation de Scikit-learn intègre déjà certaines étapes de normalisation, notamment la conversion en minuscules, et les résultats expérimentaux ont montré que ce prétraitement supplémentaire n'apportait pas d'amélioration significative des performances. Finalement nous avions une pipeline 1 avec les convertions en minuscules, suppression des accents, stopwords, etc qui a été abandonnée. À la place nous utilisons la TF-IDF directement.

3 Classification

Dans cette section, nous présentons les différentes méthodes de classification testées sur nos jeux de données. Nous comparons leurs performances afin d'identifier les approches les plus efficaces pour chaque tâche.

3.1 Méthodologie

Nous utilisons une représentation TF-IDF avec des n-grammes (1,2). Les modèles sont évalués à l'aide d'une validation croisée (K-Fold) afin d'obtenir des résultats robustes.

Pour le jeu de données des films, nous utilisons une validation croisée à 5 plis, tandis que pour les discours politiques nous utilisons une validation croisée à 10 plis.

Les métriques utilisées sont :
- Accuracy
- F1-score
- Précision
- Rappel
- ROC-AUC (pour les discours)

3.2 Modèles testés

Nous avons expérimenté plusieurs modèles de classification :
- SVM linéaire (`LinearSVC`)
- Régression logistique
- Naive Bayes multinomial
- Transformers (BERT)

3.2.1 Classification simple

Lors des premières experimentations, on a testé dans la partie bonus du TP3a un Word2Vec avec une Regression Linéaire pour faire la classification sur Chirac/Mitterrand. En remplaçant, le Word2Vec par une TF-IDF, on a ensuite continué de tester des modèles de classification différents qui sont la Regression Linéaire, la SVM linéaire, la Random Forest, la SVM avec noyau rbf et la SVM avec noyau poly. Après avoir testé ces différents modèles, on a trouvé que les modèles plus complexes type SVM avec noyau rbf ou random forest fessaient du sur-apprentissage très facilement et que les meilleurs résultats étaient obtenus avec des modèles plus simples (SVM linéaire, Regression Linéaire, Naive Bayes).

3.2.2 Transformers

Le transformer est la méthode la plus avancée que nous avons testée mais aussi celle que nous avons le moins apréciée en raison de sa complexité et de son coût en calcul. Pour le dataset de films le modèle qui a été choisi est le roberta-base. Ce modèle est fait pour des taches de classification de texte. Notre tache a été de fine-tuner le modèle, pour cela nous avons utilisé un tokenizer avec une taille de 512 tokens, un learning rate de $1.5e-5$ un batch size de 32. Les transformers sont adaptées pour les tâches avec de grandes données équilibrées et qui comportent des données corrélées. Cela explique les bons résultats que nous avons obtenus sur le dataset de films.

Contrairement aux analyses de films nous avons utilisé camembert-base comme modèle qui est adapté pour le français. Nous avons utilisé un tokenizer avec une longueur maximale de 128 tokens car les discours sont juste des phrases donc plus courtes que les films. Nous avons choisi adamW comme optimiseur avec un learning rate de $2e-5$. Le point le plus important est la Cross Entropy avec poids dynamique pour compenser le déséquilibre des classes et donc possiblement compenser le biais. De plus la classification de présidents est plus complexe que celle des films car la différence est dans le style de parole avec souvent un vocabulaire commun.

Pour les deux transformers nous avons essayé plusieurs learning rate, nombre d'epochs et aussi ajouté du dropout. Nous avons aussi essayé d'undersampler le dataset de discours de présidents.

3.3 Résultats et analyse

3.3.1 Résultats sur les 3 classificateurs (SVM linéaire, MultinomialNB et LogisticRegression)

Pour les classificateurs SVM linéaire, MultinomialNB et LogisticRegression, on observe des temps de calcul extrêmement faibles (de l'ordre de la seconde). Cela nous a permis de tester un grand nombre de combinaisons de paramètres.

tf-idf +	SVM linéaire	MultinomialNB	LogisticRegression
Accuracy	88.750% (± 1.107%)	84.400% (± 1.338%)	86.300% (± 1.259%)
F1-score	88.673% (± 1.150%)	84.128% (± 1.543%)	86.307% (± 1.342%)
Précision	89.156% (± 0.816%)	85.571% (± 2.544%)	86.151% (± 2.487%)
Rappel	88.214% (± 1.933%)	82.938% (± 3.987%)	86.510% (± 1.242%)

Table 3 – Résultats sur les commentaires de films

Commentaires de films On observe de meilleurs résultats avec le SVM linéaire (88.75%) par rapport à la régression logistique (86.30%) et au Naive Bayes (84.40%). Le SVM linéaire est particulièrement adapté à ce type de tâche car il cherche à maximiser la marge entre les classes dans un espace de grande dimension, ce qui est cohérent avec une représentation TF-IDF qui produit des vecteurs creux mais de très haute dimension. La régression logistique donne de bons résultats également, mais elle est plus sensible aux features corrélées, ce qui peut expliquer l'écart. Le naive bayes supose que les features sont indépendantes, ce qui n'est pas le cas.

tf-idf +	SVM linéaire	MultinomialNB	LogisticRegression
F1-score	94.815% (± 0.239%)	92.987% (± 0.242%)	93.943% (± 0.260%)
ROC AUC	89.704% (± 0.545%)	74.189% (± 0.988%)	87.800% (± 0.778%)
Avg Precision	98.018% (± 0.210%)	94.606% (± 0.437%)	97.634% (± 0.265%)

Table 4 – Résultats sur Chirac/Mitterrand

Chirac / Mitterrand Les résultats du tableau Chirac/Mitterrand sont biaisés par le déséquilibre du dataset (notamment l'Average Precision très élevée par rapport au leaderboard).

On observe néanmoins que les meilleurs résultats sont obtenus avec le SVM linéaire et la régression logistique. Cependant, le SVM linéaire ne permet de prédire que les classes, et non leurs probabilités, contrairement à la régression logistique. Cela permet d'améliorer les performances via un post-traitement uniquement avec la régression logistique.

Le post-traitement gaussien ne fonctionne pas bien dans nos tests, en raison du déséquilibre du dataset qui biaise fortement les résultats.

Trois soumissions ont été effectuées :

1. SVM linéaire
2. LogisticRegression
3. LogisticRegression + post-traitement

tf-idf +	SVM linéaire	MultinomialNB	LogisticRegression
F1-score	62.091%	58.549%	82.066%
ROC AUC	77.615%	88.028%	97.618%
Avg Precision	44.018%	63.733%	89.431%

Table 5 – Résultats sur Chirac/Mitterrand

3.3.2 Transformer

Pour le transformer, nous avons fait une soumission avec le roberta-base pour les films.

	F1-score	Accuracy	Precision	Recall
Films	94.381%	94.452%	95.601%	93.192%

Table 6 – Résultats sur Films avec transformer

Le transformer a donné de très bons résultats sur le detaset de films mais les résultats sur les discours de présidents sont décevants. Nous n'avons pas réussi à gérer le déséquilibre des classes et le modèle a tendance à prédire la classe majoritaire (Chirac) pour la grande majorité des phrases. Nous nous sommes aussi heurtés à des problèmes de surapprentissage. Une amélioration possible serait de faire de la data augmentation pour le jeu de données des discours de présidents.

4 Conclusion

Ce projet nous a permis d'explorer différentes approches de traitement automatique du langage naturel, en combinant des étapes de prétraitement, de représentation des données et de classification.

Nous avons constaté que la TF-IDF avec des modèles linéaires comme les SVM et la régrssion logistique donne des résultats satisfaisants surtout quand elles sont combinées avec des techniques de post-traitement pour les datasets déséquilibrés. Nous avans aussi vu que les modèles plus complexes comme les transformers peuvent donner de très bons résultats sur des tâches relativement simples et équilibrées, mais peuvent être moins performants sur des tâches plus complexes et déséquilibrées, surtout si le dataset est petit.

L'étude du jeu de données Chirac/Mitterrand a mis en évidence les limites des approches standards face au déséquilibre des classes.

Comme perspectives, il serait intéressant d'explorer des techniques de rééquilibrage des données (oversampling, data augmentation) afin d'améliorer les performances sur des jeux de données déséquilibrés et complexes.